

부록 A. 용어집

이 책은 AI 에이전트 개발이나 Agent-Evaluator SDK를 접해본 적 없는 개발자도 읽을 수 있도록 썼다 — 그래도 각 장이 처음 등장하는 용어를 매번 처음부터 다시 설명하지는 않는다(같은 설명이 반복되면 오히려 읽기 피곤해진다). 대신 이 부록에 한자리에 모아 둔다. `00_Book_forge_둘러보기.md` § 0.0이 가장 자주 쓰이는 여섯 단어(LLM·프롬프트·환각·RAG·데코레이터·Harness)를 먼저 짚어주므로, 아직 안 읽었다면 그쪽을 먼저 보는 편이 낫다 — 이 부록은 그보다 범위가 넓은, 책 전체에서 산발적으로 등장하는 용어를 담당한다.

각 항목은 "이 책에서 그 단어가 정확히 무엇을 가리키는가"만 답한다 — 일반적인 AI 업계 정의와 Book-forge 코드가 실제로 구현한 의미가 다르지 않도록, 가능한 곳마다 실제 소스 파일을 함께 표기했다.

A.1 AI/LLM 기본 개념

용어	정의	담당 챕터
토큰(token)	LLM이 텍스트를 처리하는 최소 단위 — 단어 하나가 아니라 그보다 잘게 쪼개진 조각(음절·어절 일부)인 경우가 많다. "토큰 예산"(<code>num_predict</code> 등)은 한 번의 응답 생성에 LLM이 쓸 수 있는 토큰 개수 상한을 뜻한다.	1장
시스템 프롬프트 (system prompt)	프롬프트 중에서도 "이 LLM이 어떤 역할을 맡아야 하는가"를 지시하는 부분. Book-forge의 <code>generate(prompt, system=...)</code> 가 이 둘을 인자로 분리해 받는다.	1~2장
추론 모델 (thinking/reasoning model)	최종 답을 내기 전에 "생각 과정"을 별도로 만들어내는 LLM(<code>qwen3.6:35b-mix</code> 등). 이 사고 과정이 토큰 예산을 다 써버리면 최종 응답이 빈 채로 끝나는 실패가 실제로 있었다 — Book-forge는 <code>think: False</code> 로 이를 강제로 끈다.	1·3장
임베딩(embedding)	텍스트를 숫자 벡터로 변환한 것 — 의미가 비슷한 문장일수록 벡터 공간에서 더 가까이 위치한다. Book-forge는 이 성질을 이용해 "질문과 의미상 가장 가까운 소스 조각"을 검색한다(RAG의 핵심 메커니즘).	7장

용어	정의	담당 챕터
코사인 유사도 (cosine similarity)	두 임베딩 벡터가 "얼마나 같은 방향을 가리키는가"로 의미적 유사도를 계산하는 방법. Book-forge는 별도 벡터 DB 없이 <code>numpy</code> 행렬 연산 한 줄로 이를 계산한다.	7장
청크(chunk)	RAG가 검색 단위로 쪼갠 텍스트 조각. 소스 문서 하나를 통째로 프롬프트에 넣는 대신, 관련성 높은 청크 몇 개만 골라 넣는 것이 RAG의 기본 동작이다.	7장
환각(hallucination)	LLM이 근거 없는 내용을 사실처럼 자신 있게 답하는 현상. <code>00_Book_forge_돌러보기.md</code> § 0.0 참고.	3·9·10장
RAG(Retrieval-Augmented Generation, 검색 증강 생성)	관련 자료를 먼저 검색해 프롬프트에 근거로 넣어주는 기법. <code>00_Book_forge_돌러보기.md</code> § 0.0 참고.	7장
프롬프트 인젝션(prompt injection)	외부에서 가져온 텍스트(RAG 소스, 웹 검색 결과 등) 안에 "LLM에게 다른 지시를 내리려는" 문구가 섞여 들어오는 공격/위험. <code>ChapterDrafterAgent</code> 가 <code>ThreatSeverityConfig</code> 를 쓰는 이유가 이것이다.	8장

A.2 소프트웨어 패턴

용어	정의	담당 챕터
데코레이터 (decorator)	함수를 감싸 부가 기능을 덧붙이는 파이썬 문법. <code>00_Book_forge_돌러보기.md</code> § 0.0 참고.	2장
팩토리 패턴 (factory pattern)	객체(여기서는 데코레이터가 적용된 함수)를 미리 만들어두지 않고, 필요한 시점에 필요한 값(<code>llm</code> , <code>monitor</code> 등)을 받아 그때 만드는 패턴. Book-forge의 모든 에이전트가 <code>build_X(llm, monitor)</code> 형태로 이 패턴을 쓴다 — <code>monitor</code> 가 프로젝트마다 달라 지므로 모듈을 불러오는 시점에 미리 고정할 수 없기 때문이다.	2장
Protocol (구조적 타이핑)	파이썬 <code>typing.Protocol</code> — 어떤 클래스가 "이 이름의 메서드를 이 시그니처로 갖고 있는가"만으로 호환 여부를 판단하는 타입 정의 방식(상속 관계를 요구하지 않는다). Book-	1장

용어	정의	담당 챕터
	forge의 <code>LLM</code> Protocol이 OpenAI/Anthropic/Ollama 세 구현체를 같은 인터페이스로 묶는 데 이 방식을 쓴다.	
부작용 (side effect)	함수가 값을 반환하는 것 외에 상태를 바꾸는 동작(파일 쓰기, 네트워크 요청 등). Book-forge는 "부작용 없는 함수"(LLM 응답 반환)와 "부작용 있는 함수"(파일 쓰기)를 완전히 다른 두 축(<code>@agent_eval</code> vs <code>@tool_guard</code>)으로 다룬다 — 이 구분이 이 책 3~4부 전체의 핵심 축이다.	8·1 2장

A.3 Agent-Evaluator / Harness Engineering 개념

Harness Config 개별 항목의 전체 목록(어느 에이전트가 무엇을 쓰는지)은 8장 (§ 8.1)을 참고하라. 이 절은 그 개별 항목들이 속하는 더 큰 개념만 정의한다.

용어	정의	담당 챕터
Harness Engineering	AI 에이전트 실행 결과를 계측·판정하거나 위험한 동작을 사전에 막는 장치 전체를 가리키는 이름. <code>00_Book_forge_둘러보기.md</code> § 0.0 참고.	3~4부 전체
Gate A-G	에이전트 결과물을 채점하는 7개 축(목표 달성·행동 무결성·신뢰성·성능·보안·다중 에이전트·관측성).	9장
Tracker	데이터를 실제로 쌓고 계산하는 객체. <code>PerformanceMonitor</code> 가 만들어지는 순간 전부 자동으로 생성되며, 켜고 끌 수 없다 — Config가 하나도 없어도 이미 동작하고 있다. <code>LatencyTracker</code> · <code>TaskCompletionTracker</code> · <code>AgentCoordinationTracker</code> 등.	9장 (§ 9.1)
Harness Config	<code>GoalAlignmentConfig</code> · <code>SLAConfig</code> 등, 특정 Gate/Tracker의 채점 기준을 에이전트별로 조정하는 설정 객체(dataclass) — Tracker와 달리 옵트인이다. Agent-Evaluator SDK 전체는 33개를 제공하며, Book-forge는 그중 일부만 골라 쓴다.	8장, 9장 (§ 9.1)

용어	정의	담당 챕터
<code>@agent_eval</code>	LLM 호출 함수를 감싸 Harness Config대로 Gate 점수를 계산·기록하는 배치(사후) 평가 데코레이터.	2장
<code>conversation_eval</code>	<code>@agent_eval</code> 과 달리 대화 대화 세션 전체를 하나의 단위로 채점하려는 목적의 데코레이터. Book-forge는 이걸 쓰지 않는 이유를 6장에서 직접 SDK 소스를 근거로 설명한다.	6장
<code>PerformanceMonitor</code>	한 프로젝트(책) 안에서 계측 결과를 <code>eval_results/*.json</code> 에 쌓고, 여러 챕터 결과를 <code>.merge()</code> 로 병합하는 객체.	9·11장
<code>TaskResult</code>	<code>@agent_eval</code> 이 호출 한 번마다 기록하는 채점 단위. <code>LoopDetectionConfig</code> 같은 반복 탐지가 작동하려면 각 라운드가 독립된 <code>TaskResult</code> 여야 한다 — 이것이 6장이 <code>conversation_eval</code> 대신 라운드별 <code>@agent_eval</code> 을 쓰는 이유다.	6장
<code>HallucinationDetector</code>	<code>rag_mode=True</code> 인 에이전트에서 자동 활성화되는, 근거 없는 서술을 잡아내는 Gate C 하위 채점기.	7·9장
<code>LiveGuardrail</code> / <code>@tool_guard</code>	파일 쓰기처럼 되돌리기 어려운 동작을 실행 전에 막는 실시간 측. <code>@agent_eval</code> (사후 채점)과 완전히 다른 측이다.	12장
<code>LoopDetectionConfig</code>	같은 호출(또는 같은 피드백)이 연속으로 반복되는 것을 탐지하는 Harness Config. 배치 평가(<code>@agent_eval</code>)와 실시간 가드레일(<code>LiveGuardrail</code>) 양쪽에서 모두 쓰인다.	6·12장
<code>ScopeConfig</code>	도구 이름(함수 이름) allow/forbid 목록을 검사하는 설정 — 파일 경로를 검사하는 기능이 아니다(자주 오해되는 지점). "프로젝트 디렉토리 밖 쓰기 금지" 같은 경로 검사는 Book-forge가 직접 구현한다.	3·12장
<code>TeamConcurrencyConfig</code>	여러 저자가 동시에 같은 파일을 저장하려 할 때 충돌을 감지하는 설정. 웹 에디터가 이를 이용해 저장을 HTTP 409로 차단한다.	13장

A.4 Book-forge 고유 개념

용어	정의	담당 챕터
지식창고 (Knowledge Store)	집필 세션이 수집한 RAG 소스(청크 + 임베딩)를 프로젝트 디렉토리에 영속화하는 JSON 파일(<code>knowledge/store.json</code>). 집필 에이전트와 대화 에이전트가 이 파일을 하나로 세션을 넘나들며 간접 협업한다.	7장
스캐폴딩 (scaffolding)	목차가 확정된 뒤, 실제 본문이 없는 빈 챕터 파일들을 미리 만들어두는 단계.	4·12 장
목차 매니페스트	<code>01_목차.md</code> 안에 사람이 읽는 마크다운과 함께 담기는 <code>````toc</code> 코드 블록 — 코드가 파싱해 실제 챕터 구조를 만드는 데 쓰는 부분.	4장
정적 검증기 (static verifier)	LLM을 전혀 호출하지 않고 결정론적 규칙으로 결과물을 대조하는 검증 도구 (<code>code_consistency_checker.py</code> 등). Gate 점수와는 별도 축이다.	10장
챕터 드리프트 (drift)	배치로 여러 챕터를 재생성할 때, 서로 무관했던 챕터들이 같은 소스 내용으로 수렴해버리는 현상.	3·7 장
승인 루프 (review loop)	저자가 Enter(승인)를 누르거나 수정 요청을 입력해, LLM이 그 피드백을 반영해 다시 쓰는 반복 과정.	6장

이 부록에도 없는 용어를 만났다면, 그 용어가 처음 등장하는 장의 "참고 자료" 절이 가리키는 실제 소스 파일을 직접 열어보는 것이 가장 정확하다 — 이 책의 원칙이기도 하다 (`00_기획안.md` § 2).